

	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 2	Fecha 2026-sep-15
---	---	-----------------------	----------------------

Índice

[Presentado por](#)

[Introducción](#)

[Solución](#)

- [1. Archivos Relevantes en el .ZIP](#)
- [2. Componentes](#)
- [3. Modelo Threads + Mutex](#)
- [4. Decisiones de diseño](#)
- [5. Seguridad y consistencia](#)
- [6. Liveness y escalabilidad](#)
- [7. Validación](#)
- [8. Transiciones de Estado](#)
- [9. Snapshot](#)
- [10. Conclusión](#)

Presentado por

Andrés Felipe Peña Castro (código 201618814)

Introducción

Esta tarea implementa un sistema de venta y reserva de asientos a partir del skeleton proporcionado, incorporando el modelo de concurrencia `Threads + Mutex`. La solución mantiene las reglas de negocio separadas del mecanismo de sincronización, permitiendo que las operaciones sobre cada sector se ejecuten de forma segura y concurrente. Este informe describe las principales decisiones de diseño y explica cómo la implementación satisface los requisitos establecidos en `project1_spec.pdf`.

Solución

Se divide en tres partes. `Gate.Venue` es la fachada de la API y dirige las operaciones hacia `Gate.Sync`. Este concentra la gestión de concurrencia mediante un proceso independiente por sector, que serializa las operaciones sobre su estado. Finalmente, `Gate.Sector` y `Gate.Row` contienen las reglas de negocio relacionadas con reservas, asientos, holds, confirmaciones y cancelaciones, sin depender del mecanismo de concurrencia. Esta separación permite mantener la lógica del dominio independiente del modelo `Threads + Mutex` utilizado en esta implementación.

	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 2	Fecha 2026-sep-15
---	--	------------------------------	-----------------------------

1. Archivos Relevantes en el .ZIP

ISIS4218_T02.zip

```

.... reporte_tarea_2_cpd.pdf
.... README.md
.... .gate_model -> para esta primera tarea es "threads"
.... a1_threads/
..... lib/
..... gate/

..... api.ex
..... row.ex
..... sector.ex -> ligeramente modificada para tipado
..... sync.ex -> implementación principal de esta tarea
..... venue.ex
..... venue_spec.ex

```

2. Componentes

La solución se organiza en módulos con responsabilidades diferenciadas:

1. `Gate.API` define la interfaz que debe cumplir la implementación y establece las operaciones disponibles para interactuar con el sistema.
2. `Gate.Venue` actúa como fachada de la API. Recibe las operaciones y las dirige hacia la capa de sincronización, sin implementar directamente la lógica de concurrencia.
3. `Gate.Sync` mantiene el estado de cada sector dentro del proceso que lo administra. El estado no se comparte entre procesos, por lo que las operaciones externas deben pasar por **Gate.Sync.update/3**.
4. `Gate.Sector` contiene las reglas de negocio de cada sector, incluyendo reservas, confirmaciones, cancelaciones, holds y disponibilidad.
5. `Gate.Row` proporciona las operaciones relacionadas con las filas y sus asientos.
6. **Separación de responsabilidades:** la lógica de negocio se mantiene independiente del mecanismo de concurrencia. Esto permite cambiar la estrategia de sincronización sin tener que modificar las reglas de ticketing.

3. Modelo Threads + Mutex

La implementación utiliza un proceso independiente para cada sector. Cada proceso mantiene la propiedad exclusiva del estado de su sector y recibe las operaciones mediante su mailbox. Las operaciones son procesadas de manera secuencial dentro del proceso, lo que proporciona la exclusión mutua necesaria para evitar actualizaciones concurrentes sobre el mismo estado.

	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 2	Fecha 2026-sep-15
---	--	------------------------------	-----------------------------

`Gate.Sync.update/3` centraliza este mecanismo: envía la operación al proceso correspondiente, espera su resultado y permite que la función se ejecute sobre el estado actual como una única actualización. Al existir un proceso independiente por sector, las operaciones sobre sectores diferentes pueden ejecutarse de manera independiente.

- `Sync.start/1` crea un proceso por sector.
- El estado del sector es propiedad exclusiva de ese proceso.
- `update/3` envía una función, una referencia y el proceso solicitante.
- El proceso ejecuta la función sobre el estado actual y solo después procesa el siguiente mensaje.
- La función recibe el estado actual y produce un `new_state`, que se convierte en el estado de la siguiente iteración.
- La operación completa se ejecuta de forma indivisible sobre el sector, evitando que otra operación lea o modifique el estado mientras está siendo actualizado.
- La referencia única permite asociar cada respuesta con la operación que la originó.

4. Decisiones de diseño

La concurrencia se aisló en `Gate.Sync` para separar el mecanismo de sincronización de las reglas de negocio. Este módulo es responsable de administrar los procesos por sector, recibir las operaciones y garantizar que las actualizaciones sobre el estado se ejecuten de forma atómica.

`Gate.Sector` permanece como un módulo puro, encargado únicamente de las reglas de ticketing y las transiciones de estado. De esta forma, no depende de procesos, mensajes ni mecanismos de sincronización, lo que facilita su validación y reutilización.

Esta separación también permite cambiar el modelo de concurrencia en las siguientes tareas sin modificar las reglas de negocio. En cada implementación puede reemplazarse `Gate.Sync` por el mecanismo correspondiente, mientras `Gate.Sector` y la interfaz pública de `Gate.Venue` permanecen esencialmente iguales.

5. Seguridad y consistencia

La sincronización se realiza por sector mediante un proceso que mantiene el estado y procesa las operaciones de forma secuencial. Esto evita que dos operaciones concurrentes modifiquen simultáneamente los mismos asientos, previniendo *double-sell* y *lost updates*. Al centralizar las transiciones de estado también se preservan estados válidos del inventario. Finalmente, *cada hold recibe un id único* que no se reutiliza durante la vida del venue.

Ejemplos que resuelve el código entregado:

- Double-sell:** Si dos reservas concurrentes intentan asignar los mismos asientos, ambas solicitudes llegan al proceso propietario del sector, pero se procesan una después de la otra. La primera marca los asientos como `:held`; cuando se procesa la segunda, encuentra el estado actualizado y esos asientos ya no están disponibles.

	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 2	Fecha 2026-sep-15
---	--	------------------------------	-----------------------------

- b. **Lost updates:** Cada operación recibe el estado actual del sector y produce un nuevo estado completo. Este nuevo estado se convierte en el estado utilizado por la siguiente operación, por lo que una actualización no puede sobrescribir los cambios realizados por otra operación concurrente.
- c. **Estados inválidos:** Las transiciones de los asientos se realizan de forma controlada: una reserva cambia `:free` a `:held`, una confirmación cambia `:held` a `:sold` y una cancelación o expiración cambia `:held` a `:free`. Al procesarse estas transiciones secuencialmente dentro del sector, no existen modificaciones simultáneas que puedan producir estados inconsistentes.
- h. **Hold confirmado que vuelve a liberar sus asientos:** Una vez confirmado, el hold pasa a `:confirmed` y sus asientos a `:sold`. Además, se elimina de `pending`, por lo que ni `cancel` ni `sweep` pueden volver a liberar esos asientos.
- d. **Hold cancelado que vuelve a confirmarse:** `confirm` solo permite completar la operación cuando el hold está en estado `:held`. Si ya fue cancelado, su estado es `:cancelled` y la operación no puede volver a vender esos asientos.
- e. **Holds únicos:** Cada reserva exitosa obtiene un identificador mediante un contador monótono por sector. Como las entradas de `sector.holds` no se eliminan, un identificador utilizado anteriormente no puede reutilizarse para representar otro hold.
- f. **Expiración segura:** `sweep` se ejecuta antes de `reserve`, `confirm`, `cancel`, `availability` y `snapshot`. Los holds cuyo TTL venció pasan a `:expired`, sus asientos vuelven a `:free` y dejan de pertenecer a `pending`. La expiración se realiza de forma lazy, sin mantener un proceso independiente dedicado a revisar los holds.
- g. **All-or-nothing:** En `:any`, los asientos pueden seleccionarse entre diferentes filas, mientras que `:contiguous` exige encontrar un bloque consecutivo dentro de una misma fila. La asignación se construye antes de actualizar el estado, por lo que si no se puede completar la cantidad solicitada no se conserva ninguna asignación parcial.

6. Liveness y escalabilidad

La implementación evita `deadlocks` porque cada operación se procesa de forma secuencial dentro del proceso responsable de su sector, sin adquirir múltiples mecanismos de bloqueo. Las operaciones no esperan a que haya disponibilidad de asientos ni implementan colas o reintentos para obtenerlos; cada solicitud se evalúa una sola vez sobre el estado disponible al ser procesada. Además, cada sector mantiene su propio proceso, permitiendo que las operaciones sobre sectores diferentes se ejecuten de manera independiente. Como limitación, las operaciones de un mismo sector se serializan, por lo que la concurrencia efectiva dentro de un sector está limitada por un único proceso.

Limitación principal: que las operaciones dentro de un mismo sector se serializan en un único proceso, por lo que un sector altamente concurrido se convierte en un punto de contención. En contraste, sectores diferentes pueden procesarse de manera independiente.

Las pruebas de carga mostraron que la concentración de solicitudes en un mismo sector incrementa significativamente la latencia y reduce el throughput, confirmando que la serialización por sector constituye el principal punto de contención de esta implementación.

	ISIS 4218 Concurrencia, Paralelismo y Distribución	Entregable Tarea 2	Fecha 2026-sep-15
---	--	------------------------------	-----------------------------

7. Validación

La implementación fue validada mediante la compilación con `mix compile --warnings-as-errors`, la verificación de restricciones con `mix gate.check` y la ejecución de `mix test`. Adicionalmente, se realizaron pruebas de concurrencia y carga para verificar la ausencia de `double-sell`, la consistencia del estado y la independencia entre sectores bajo solicitudes concurrentes.

8. Transiciones de Estado

Operación	Estado del asiento	Estado del hold	Pending
<code>reserve</code>	<code>:free -> :held</code>	nuevo hold	se agrega
<code>confirm</code>	<code>:held -> :sold</code>	<code>:held -> :confirmed</code>	se elimina
<code>cancel</code>	<code>:held -> :free</code>	<code>:held -> :cancelled</code>	se elimina
expiración	<code>:held -> :free</code>	<code>:held -> :expired</code>	se elimina

9. Snapshot

`Gate.Sector.snapshot/2` ejecuta primero `sweep` y luego construye las listas de asientos `free`, `held` y `sold` a partir del estado del sector. Al ejecutarse dentro del proceso propietario, el resultado es consistente para ese sector. El snapshot completo del venue consulta los sectores de forma independiente, por lo que no representa necesariamente un único instante global, lo cual está permitido por la especificación.

10. Conclusión

La implementación de `Threads + Mutex` permite manejar de forma segura las operaciones concurrentes sobre los asientos, manteniendo las reglas de negocio separadas del mecanismo de sincronización. La serialización por sector garantiza consistencia y evita conflictos como `double-sell` y `lost updates`, mientras que la separación entre `Gate.Sync` y `Gate.Sector` permite mantener la lógica de ticketing independiente del modelo de concurrencia. Esta arquitectura establece una base adecuada para implementar y comparar los diferentes modelos de concurrencia requeridos en las siguientes tareas.